Evaluación de un modelo de embeddings

Relevancia semántica vs. coincidencia de palabras clave

1. Objetivo y montaje del experimento

Se quiere verificar si un modelo de embeddings recupera oraciones que **significan** lo mismo que la consulta aunque no compartan palabras, y si descarta oraciones que **comparten palabras** pero hablan de otra cosa. Para tener una línea base se compara contra TF-IDF, que solo mide coincidencia de términos.

Concepto técnico: despliegue de microservicios. **Query:** *"¿Cómo se hace el despliegue de microservicios?"*

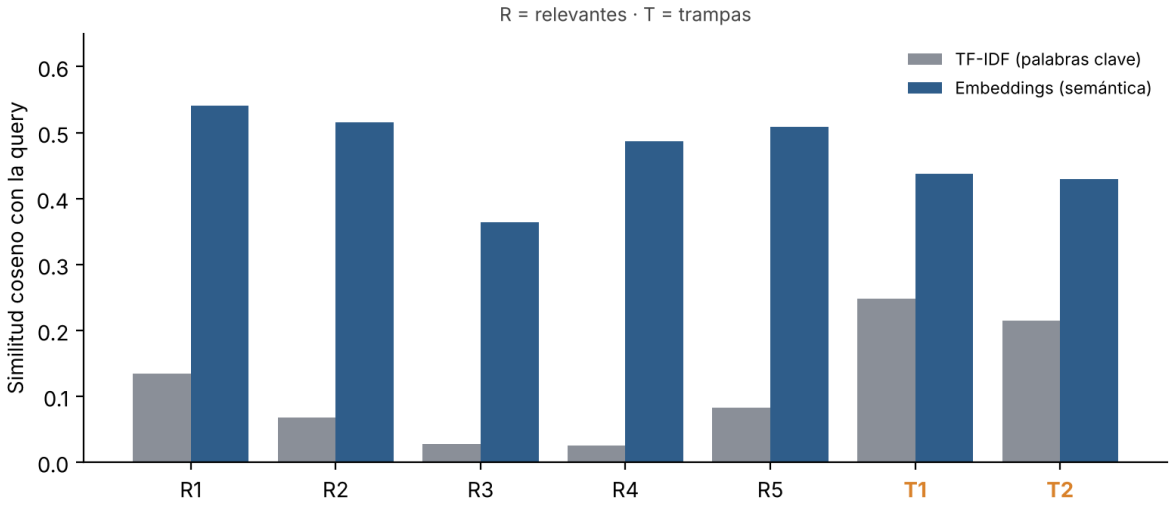
2. Conjunto de oraciones

Cinco oraciones relevantes (R) con vocabulario distinto entre sí y respecto a la query, y dos oraciones trampa (T) que comparten palabras clave pero no tienen nada que ver con microservicios.

ID	Oración	Por qué está en el conjunto
R1	Empaquetamos cada componente de la aplicación en contenedores que se actualizan de forma independiente.	Contenedores independientes = cada servicio se publica por separado.
R2	La plataforma se divide en módulos pequeños y autónomos que pasan a producción sin detener al resto.	Describe la idea central (servicios autónomos) sin usar "microservicio".
R3	Kubernetes orquesta los pods de cada módulo y los escala automáticamente según la demanda.	Vocabulario de la herramienta (Kubernetes, pods), no del concepto.
R4	Cada equipo libera su pieza del sistema por separado gracias a un pipeline de integración continua.	Enfoque organizacional: equipos que liberan su parte con CI/CD.
R5	Partimos el monolito en funciones desacopladas que llegan a los servidores dentro de imágenes Docker.	Migración del monolito a piezas desacopladas en Docker.
T1	El servicio de micro-limpieza hizo el despliegue de sus cepillos en la oficina.	Comparte "servicio", "micro" y "despliegue", pero habla de limpieza.
T2	El despliegue del paracaídas falló y el servicio de microondas del avión también.	Comparte "despliegue", "servicio" y "micro"; habla de un avión.

3. Resultados

ID	TF-IDF	Puesto	Embeddings	Puesto
R1	0.134	3	0.541	1
R2	0.068	5	0.516	2
R3	0.027	6	0.364	7
R4	0.025	7	0.487	4
R5	0.083	4	0.509	3
T1	0.248	1	0.437	5
T2	0.215	2	0.429	6



4. Análisis

TF-IDF cae en las dos trampas. T1 y T2 ocupan los puestos 1 y 2 porque son las únicas que repiten "despliegue" y "servicio". Las oraciones relevantes quedan casi en cero (0,025 a 0,134): para un buscador por palabras clave, una oración que explica microservicios sin decir "microservicio" es invisible.

Los embeddings corrigen la mayor parte del problema. R1, R2, R5 y R4 quedan en los puestos 1 a 4, todas por encima de las trampas. El modelo reconoce que "contenedores independientes", "módulos autónomos" o "monolito" pertenecen al mismo campo semántico que la query.

Hay un fallo instructivo: R3 (Kubernetes) queda de última. La oración es correcta, pero está escrita con jerga de una herramienta (Kubernetes, pods) que el modelo usado no relaciona con el concepto. Esto muestra que la calidad depende de que el modelo haya visto ese vocabulario técnico durante el entrenamiento.

El margen es estrecho. Con embeddings, la peor relevante sin contar R3 (0,487) supera a la mejor trampa (0,437) por solo 0,05. Un umbral fijo de similitud sería frágil: conviene fijarlo con datos de validación.

Sobre el modelo usado: se usaron los vectores de palabras de spaCy (*es_core_news_md*), promediados para obtener el vector de cada oración. Es un modelo de embeddings simple; un modelo de oraciones entrenado para búsqueda (p. ej. *paraphrase-multilingual-MiniLM* o *multilingual-e5*) entiende el orden y el contexto, y se esperaría que separara mejor las trampas y recuperara R3.

Conclusión: los embeddings sí discriminan entre relevancia semántica y coincidencia de palabras, y superan claramente a TF-IDF en este caso. Para producción conviene un modelo de oraciones especializado, una evaluación con más ejemplos, o una búsqueda híbrida (embeddings + palabras clave) con re-ranking.

5. Similitud coseno con scikit-learn

La similitud coseno mide el ángulo entre dos vectores, sin importar su longitud: **cos(θ) = (A·B) / (||A|| · ||B||)**. Vale 1 si apuntan en la misma dirección, 0 si son ortogonales. En *scikit-learn* se usa `sklearn.metrics.pairwise.cosine_similarity`, que recibe matrices 2D (una fila por vector) y devuelve la matriz de similitudes entre todas las filas.

Ejemplo mínimo entre dos vectores

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

a = np.array([0.2, 0.8, 0.1])
b = np.array([0.3, 0.7, 0.0])

# cosine_similarity espera matrices 2D: (n_muestras, n_dimensiones)
sim = cosine_similarity(a.reshape(1, -1), b.reshape(1, -1))[0][0]
print(f"Similitud coseno: {sim:.4f}")   # ~0.98

# Verificación manual con la fórmula
manual = a @ b / (np.linalg.norm(a) * np.linalg.norm(b))
```

Código completo del experimento

Calcula ambas similitudes (TF-IDF y embeddings) contra la query y guarda los resultados.

```
"""Compara similitud por palabras clave (TF-IDF) vs. similitud semántica (embeddings)."""
import json
import numpy as np
import spacy
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

QUERY = "¿Cómo se hace el despliegue de microservicios?"

ORACIONES = {  # R = relevantes, T = trampas
    "R1": ("Empaquetamos cada componente de la aplicación en contenedores"
           " que se actualizan de forma independiente."),
    "R2": ("La plataforma se divide en módulos pequeños y autónomos que"
           " pasan a producción sin detener al resto."),
    "R3": ("Kubernetes orquesta los pods de cada módulo y los escala"
           " automáticamente según la demanda."),
    "R4": ("Cada equipo libera su pieza del sistema por separado gracias"
           " a un pipeline de integración continua."),
    "R5": ("Partimos el monolito en funciones desacopladas que llegan a"
           " los servidores dentro de imágenes Docker."),
    "T1": ("El servicio de micro-limpieza hizo el despliegue de sus"
           " cepillos en la oficina."),
    "T2": ("El despliegue del paracaídas falló y el servicio de"
           " microondas del avión también."),
}

ids, textos = list(ORACIONES), list(ORACIONES.values())

# 1) Coincidencia de palabras clave: TF-IDF + similitud coseno
tfidf = TfidfVectorizer()
X = tfidf.fit_transform([QUERY] + textos)
sim_tfidf = cosine_similarity(X[0:1], X[1:])[0]

# 2) Similitud semántica: vectores densos (embeddings) + similitud coseno
nlp = spacy.load("es_core_news_md")
def embed(texto):
    doc = nlp(texto)
    return doc.vector  # promedio de los vectores de palabras (300 dimensiones)

q_vec = embed(QUERY).reshape(1, -1)
E = np.vstack([embed(t) for t in textos])
sim_emb = cosine_similarity(q_vec, E)[0]

res = [{"id": i, "texto": t,
        "tfidf": round(float(a), 3), "emb": round(float(b), 3)}
       for i, t, a, b in zip(ids, textos, sim_tfidf, sim_emb)]
for r in res:
    print(f"{r['id']}  TF-IDF={r['tfidf']:.3f}  Emb={r['emb']:.3f}")

with open("resultados.json", "w") as f:
    json.dump({"query": QUERY, "resultados": res}, f, ensure_ascii=False, indent=1)
```

6. Diagrama de flujo de la búsqueda semántica

Los documentos se convierten en vectores una sola vez (indexación offline). En cada consulta solo se vectoriza la query, se compara contra el índice y se devuelve el documento más parecido si supera el umbral.

